

Mount Kenya University

Unlocking Infinite Possibilities

Student Name: Abdirahman Mohamud

Admission Number: bsccs/2024/54673

Course/Class: BSc Computer Science / Year 3

Unit Code: BIT4138

Unit Name: Advanced Cryptography

Lecturer Name: Micheal Nyoro

Semester/Academic Year: Semester 2

Project Title: Classical to Block Cipher Cryptographic Toolkit

Selected Technologies: Python, VS Code, Cryptographic Simulators, Statistical Tools

Online Portfolio / GitHub Link:

<https://github.com/Abduz12/advanced/tree/main/advanced>

Contents

Week 1: Cryptography Environment Setup	4
.....	4
Fig 1: Installation of Python and VS Code	4
.....	4
Fig 2: Installation of Cryptography Libraries	4
Fig 3: OpenSSL Installation Test	5
Fig 4: Local Encryption Script Execution	6
Fig 5: GitHub Repository Initialization	6
Week 2: Classical Cryptography and Cipher Design	6
Fig 1: Python implementation of Caesar and Vigenère cipher functions....	7
Fig 2: Terminal output showing successful encryption and decryption of plaintext using both ciphers	7
Fig 3: Terminal output demonstrating a brute-force attack simulation on the Caesar Cipher	8
Week 3: Stream Ciphers and Randomness Testing	8
Fig 1: Implementation of the Linear Congruential Generator (LCG) class in Python	8
Fig 2: Terminal output displaying the pseudorandom sequence generated by the LCG	9

Fig 3: Terminal output showing the original, raw encrypted, and decrypted data using the XOR stream cipher.....	9
Week 4: Block Cipher Design and AES	9
Fig 1: Python implementation of AES key generation and encryption using the Cryptography library.....	10
Fig 2: Terminal output showing the generated Symmetric Key and successful AES encryption/decryption.....	10
Week 5: Public Key Cryptography (RSA)	10
Fig 1: Python implementation of RSA public and private key pair generation.....	10
Fig 2: Terminal output demonstrating successful encryption with the public key and decryption with the private key.....	11
Week 6: Symmetric Ciphers II – Substitution-Permutation Networks (SPN)	11
Fig 1: Python implementation of a Substitution-Permutation Network (SPN) model demonstrating S-Box dictionaries.....	12
Fig 2: Terminal output demonstrating the non-linear substitution of data to create cryptographic confusion.....	12
Week 7: Cryptanalysis	12
Fig 1: Python implementation of cryptanalytic testing, including an Avalanche Effect bit-difference calculator.....	13
Fig 2: Terminal output showing the cryptanalysis results, demonstrating significant ciphertext differences from minor plaintext alterations.....	13

Week 1: Cryptography Environment Setup

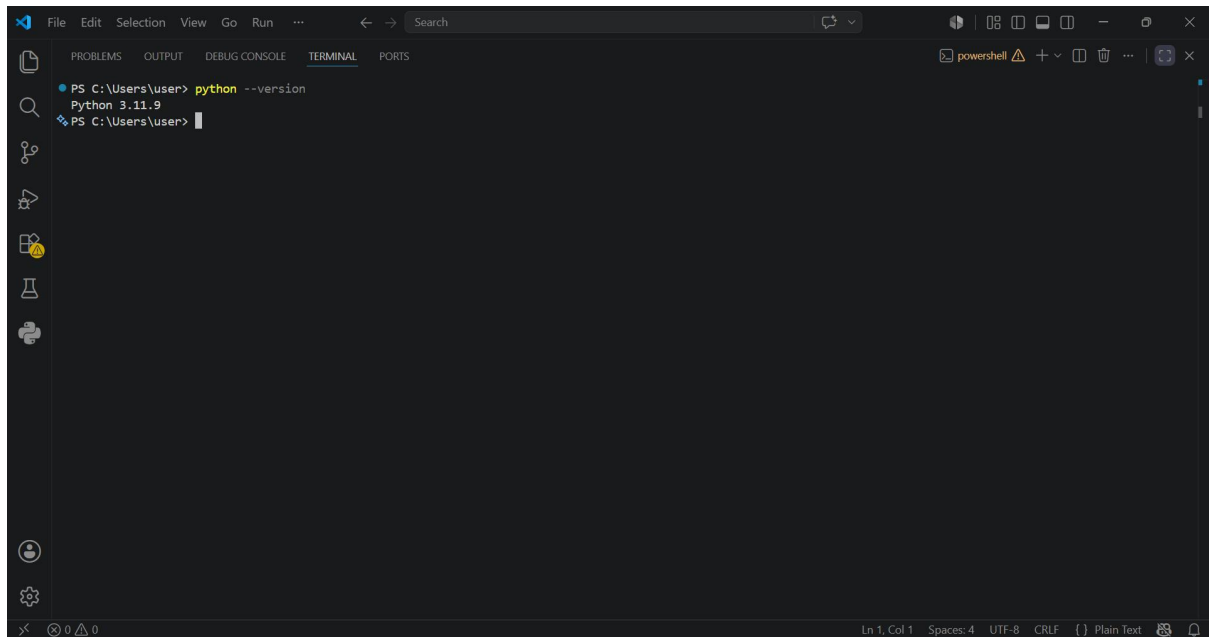


Fig 1: Installation of Python and VS Code

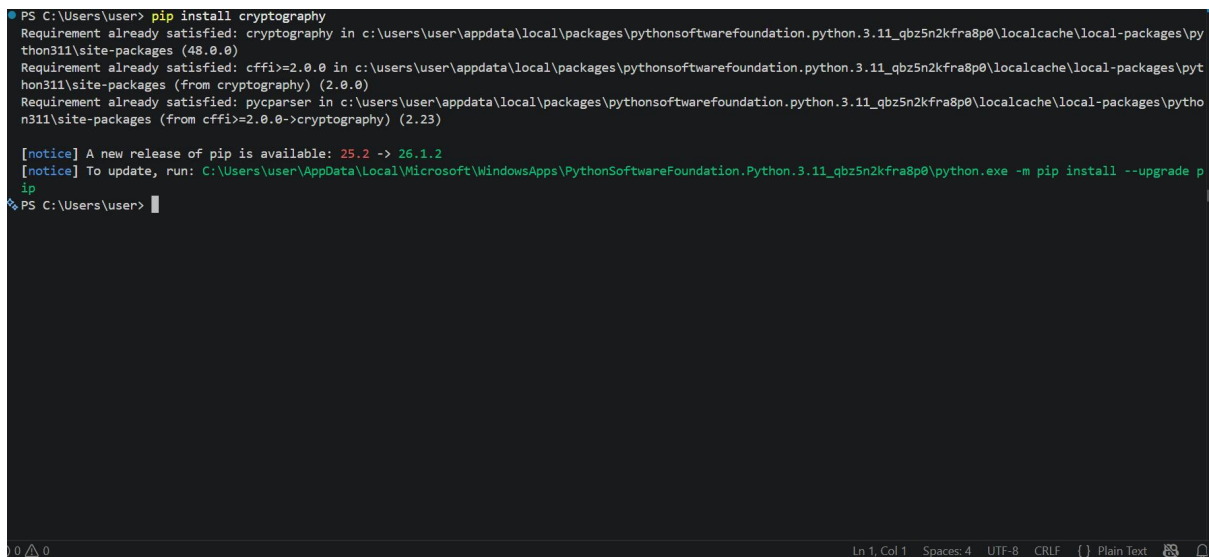
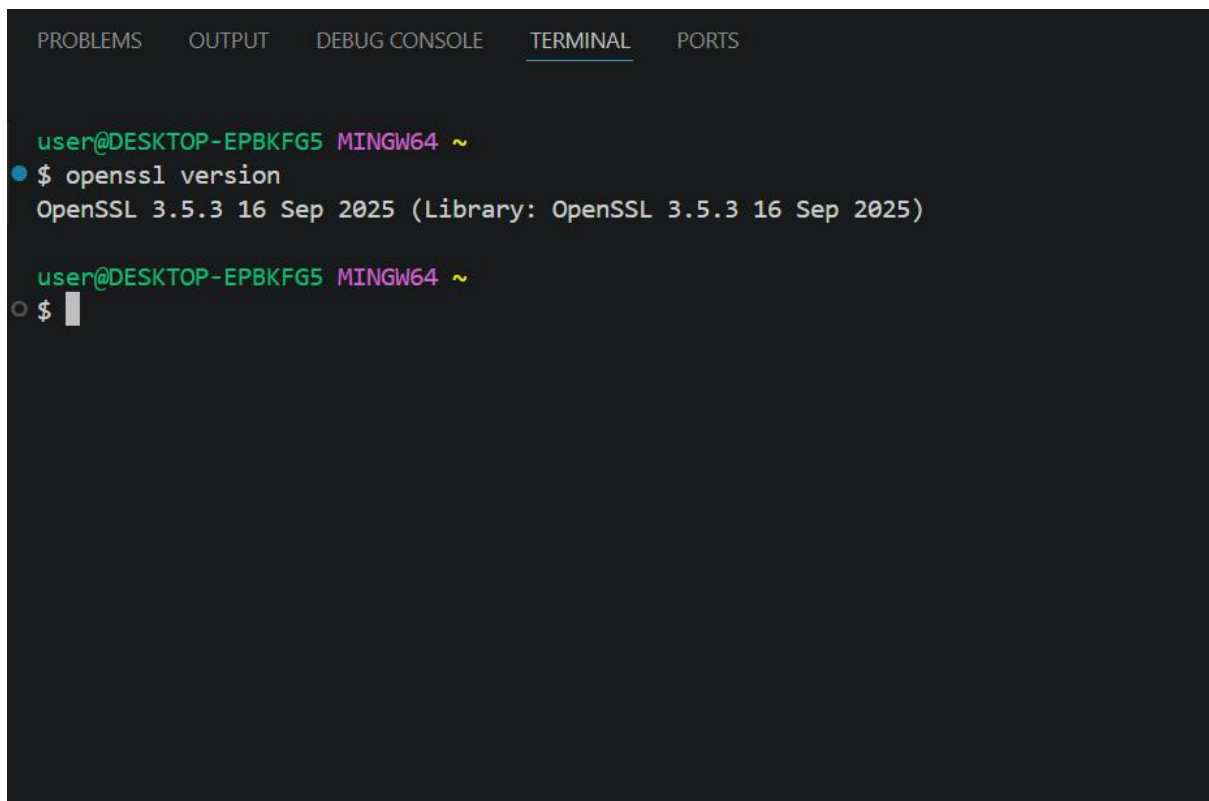


Fig 2: Installation of Cryptography Libraries

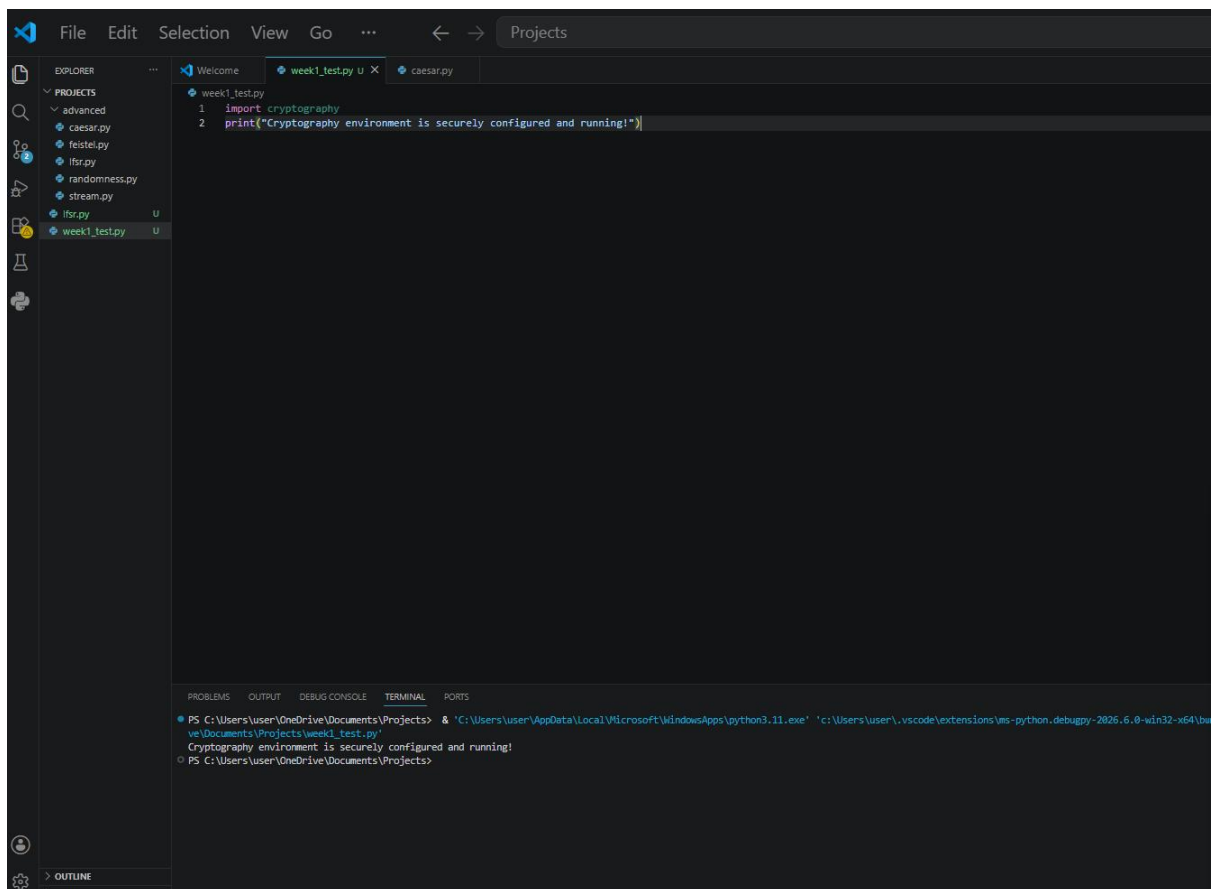


A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal shows a prompt 'user@DESKTOP-EPBKFG5 MINGW64 ~' followed by the command '\$ openssl version'. The output is 'OpenSSL 3.5.3 16 Sep 2025 (Library: OpenSSL 3.5.3 16 Sep 2025)'. Below this, the prompt appears again, followed by a '\$' and a cursor.

```
user@DESKTOP-EPBKFG5 MINGW64 ~
$ openssl version
OpenSSL 3.5.3 16 Sep 2025 (Library: OpenSSL 3.5.3 16 Sep 2025)

user@DESKTOP-EPBKFG5 MINGW64 ~
$
```

Fig 3: OpenSSL Installation Test



A screenshot of the Visual Studio Code (VS Code) interface. The Explorer sidebar on the left shows a project named 'advanced' with several files: 'caesar.py', 'feistel.py', 'itrs.py', 'randomness.py', 'stream.py', 'itrs.py', and 'week1_test.py'. The 'week1_test.py' file is selected and open in the editor. The code in the editor is as follows:

```
1 import cryptography
2 print("cryptography environment is securely configured and running!")
```

The bottom panel of the VS Code window shows the 'TERMINAL' tab. It displays the command prompt output for running the script. The command is 'PS C:\Users\user\OneDrive\Documents\Projects> & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bin\python.exe' 'c:\Users\user\OneDrive\Documents\Projects\week1_test.py''. The output is 'Cryptography environment is securely configured and running!'.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\user\OneDrive\Documents\Projects> & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bin\python.exe' 'c:\Users\user\OneDrive\Documents\Projects\week1_test.py'
Cryptography environment is securely configured and running!
PS C:\Users\user\OneDrive\Documents\Projects>
```

Fig 4: Local Encryption Script Execution

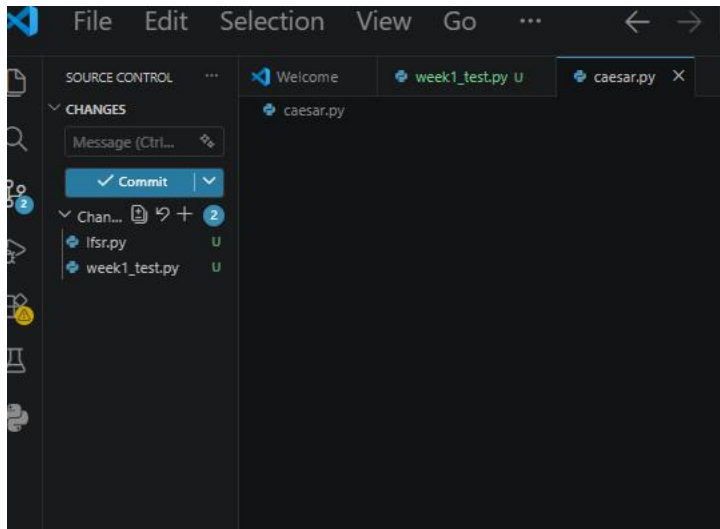


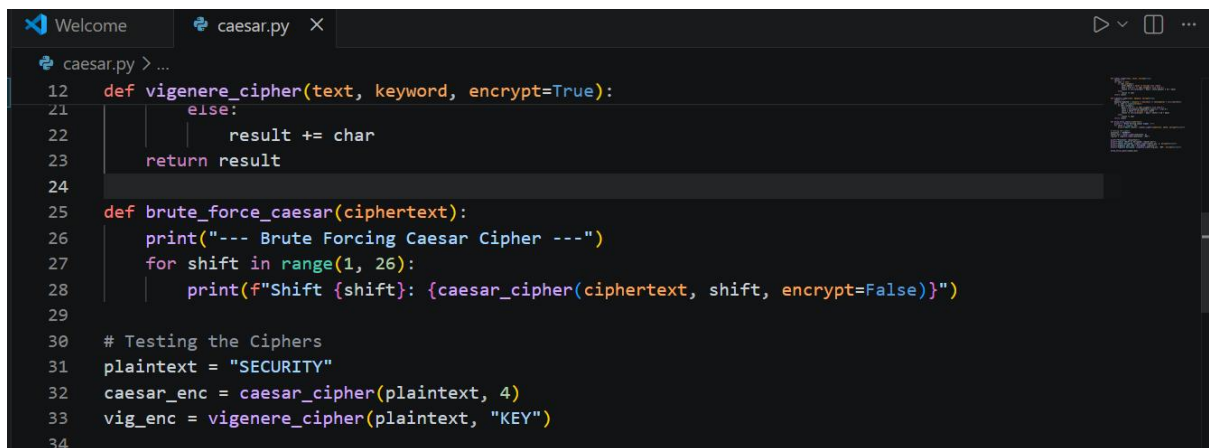
Fig 5: GitHub Repository Initialization

Student Reflection: Successfully installed and configured the cryptography development environment. Python, VS Code, and the Python Cryptography library were set up. OpenSSL was verified for secure key generation, and a GitHub repository was initialized to maintain professional version control and cybersecurity workflow reporting.

Week 2: Classical Cryptography and Cipher Design

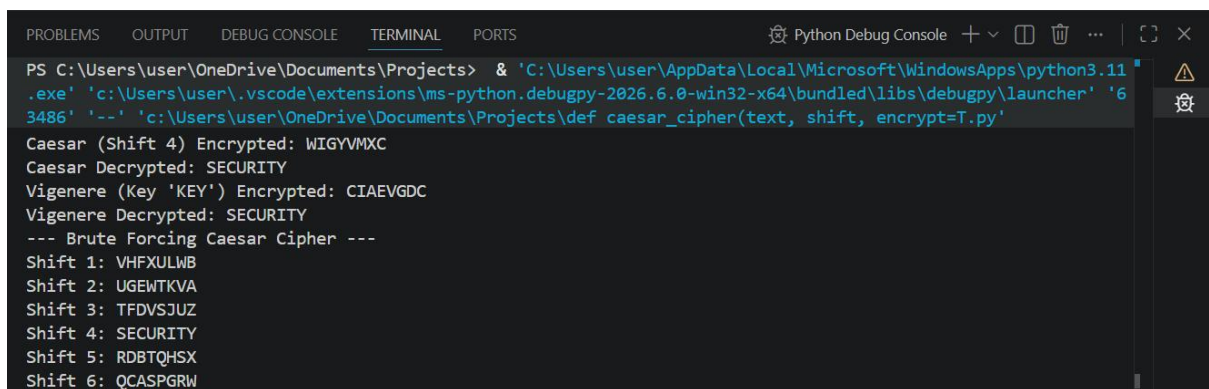
Comparison and Implementation of Caesar and Vigenère Ciphers

Implemented Caesar and Vigenère ciphers in Python. I observed that the Vigenère cipher provides enhanced security over the Caesar cipher by utilizing multiple alphabets via a keyword, which effectively reduces redundancy and makes basic frequency analysis much more difficult for attackers.



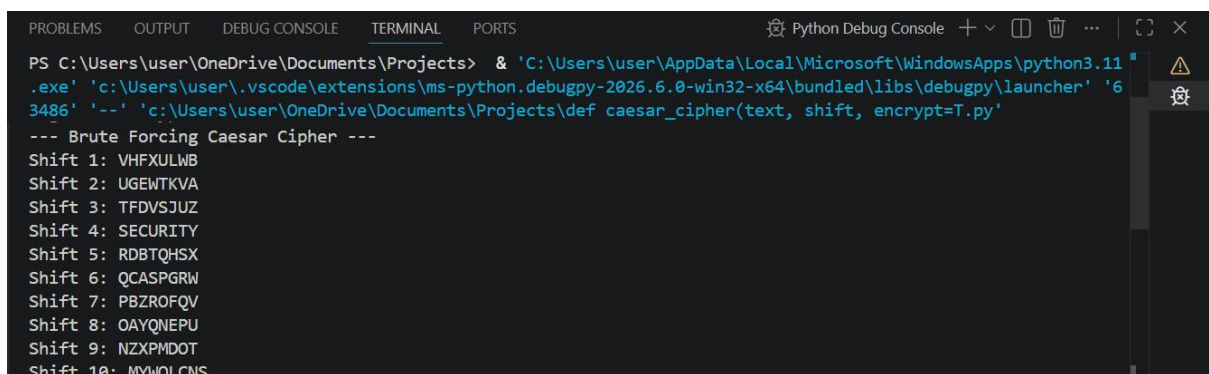
```
12 def vigenere_cipher(text, keyword, encrypt=True):
13     else:
14         result += char
15     return result
16
17 def brute_force_caesar(ciphertext):
18     print("--- Brute Forcing Caesar Cipher ---")
19     for shift in range(1, 26):
20         print(f"Shift {shift}: {caesar_cipher(ciphertext, shift, encrypt=False)}")
21
22 # Testing the Ciphers
23 plaintext = "SECURITY"
24 caesar_enc = caesar_cipher(plaintext, 4)
25 vig_enc = vigenere_cipher(plaintext, "KEY")
26
```

Fig 1: Python implementation of Caesar and Vigenère cipher functions.



```
PS C:\Users\user\OneDrive\Documents\Projects> & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11
.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '6
3486' '--' 'c:\Users\user\OneDrive\Documents\Projects\def caesar_cipher(text, shift, encrypt=T.py'
Caesar (Shift 4) Encrypted: WIGVVMXC
Caesar Decrypted: SECURITY
Vigenere (Key 'KEY') Encrypted: CIAEVGDC
Vigenere Decrypted: SECURITY
--- Brute Forcing Caesar Cipher ---
Shift 1: VHFUXLWB
Shift 2: UGEWTKVA
Shift 3: TFDVSJUZ
Shift 4: SECURITY
Shift 5: RDBTQHSX
Shift 6: QCASPGRW
```

Fig 2: Terminal output showing successful encryption and decryption of plaintext using both ciphers



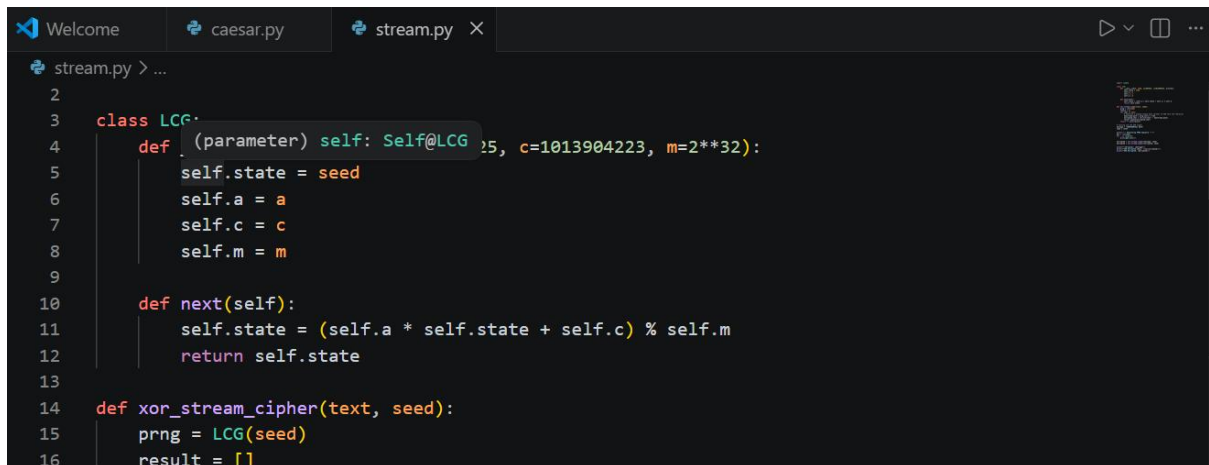
```
Shift 7: PBZROFQV
Shift 8: OAYQNEPU
Shift 9: NZXPMDOT
Shift 10: MYWQICNS
```

Fig 3: Terminal output demonstrating a brute-force attack simulation on the Caesar Cipher.

Student Reflection: Implemented Caesar and Vigenère ciphers. I observed that the polyalphabetic Vigenère cipher provides enhanced security over the Caesar cipher by utilizing a keyword, effectively reducing redundancy and making basic frequency analysis much more difficult for attackers to execute.

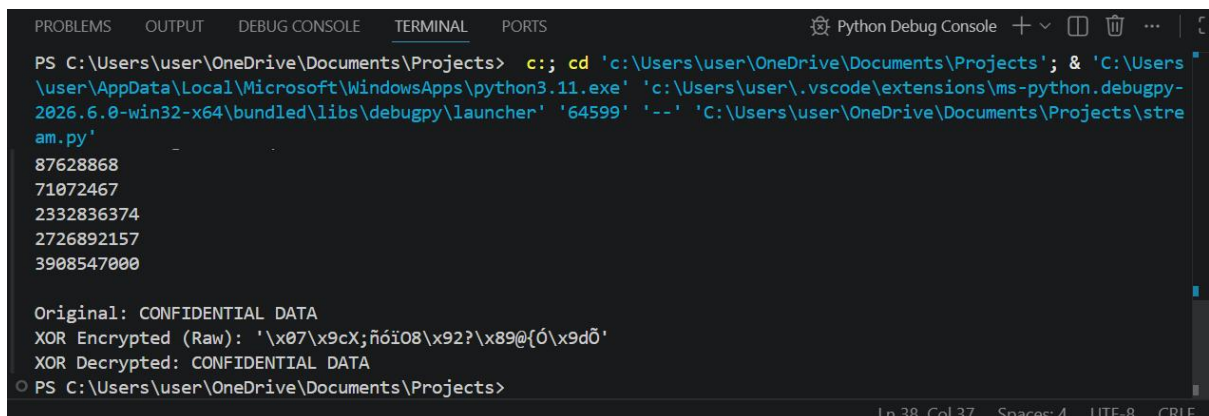
Week 3: Stream Ciphers and Randomness Testing

Title: Implementation of Stream Cipher Systems and Random Sequence Analysis



```
stream.py > ...
2
3 class LCG:
4     def __init__(self, seed, a=1, c=1013904223, m=2**32):
5         self.state = seed
6         self.a = a
7         self.c = c
8         self.m = m
9
10    def next(self):
11        self.state = (self.a * self.state + self.c) % self.m
12        return self.state
13
14    def xor_stream_cipher(text, seed):
15        prng = LCG(seed)
16        result = []
```

Fig 1: Implementation of the Linear Congruential Generator (LCG) class in Python.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console
PS C:\Users\user\OneDrive\Documents\Projects> c:: cd 'c:\Users\user\OneDrive\Documents\Projects'; & 'C:\Users\user\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\user\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '64599' '--' 'C:\Users\user\OneDrive\Documents\Projects\stream.py'
87628868
71072467
2332836374
2726892157
3908547000

Original: CONFIDENTIAL DATA
XOR Encrypted (Raw): '\x07\x9cX;ñóï08\x92?\x89@{ó\x9dõ'
XOR Decrypted: CONFIDENTIAL DATA
PS C:\Users\user\OneDrive\Documents\Projects>
```


Fig 2: Terminal output displaying the pseudorandom sequence generated by the LCG.

```
Original: CONFIDENTIAL DATA
XOR Encrypted (Raw): '\x07\x9cX;ñói08\x92?\x89@{Ó\x9dÕ'
XOR Decrypted: CONFIDENTIAL DATA
PS C:\Users\user\OneDrive\Documents\Projects>
```

Fig 3: Terminal output showing the original, raw encrypted, and decrypted data using the XOR stream cipher.

Student Reflection: Developed a Linear Congruential Generator (LCG) and applied it to an XOR stream cipher. The exercises highlighted the critical difference between true randomness and pseudorandomness, emphasizing that stream cipher security relies heavily on the unpredictability and period of its keystream.

Week 4: Block Cipher Design and AES

Title: Advanced Encryption Standard (AES) and Block Cipher Operations

The screenshot shows a VS Code editor with a dark theme. The Explorer sidebar on the left displays a project structure with folders 'advanced', 'aespy', 'caesarp', 'stream.py', and 'week1_testpy'. The 'aespy' folder is selected, and its file 'aespy.py' is open in the editor. The code in 'aespy.py' demonstrates AES encryption and decryption using Fernet. It includes comments for each step: generating a key, creating a cipher, printing the key, encrypting a plaintext, and decrypting the ciphertext. The plaintext is a string of 20 'H' characters. The ciphertext is printed as a hexadecimal string. The decrypted text is printed as a string of 20 'h' characters.

```

1 from cryptography.fernet import Fernet
2 key = Fernet.generate_key()
3 cipher = Fernet(key)
4 print(f"Generated AES Key: {key.decode()}")
5
6 plaintext = b"H" * 20
7 ciphertext = cipher.encrypt(plaintext)
8 print(f"Encrypted: {ciphertext}")
9 print(f"Decrypted: {cipher.decrypt(ciphertext).decode()}")

```

Fig 1: Python implementation of AES key generation and encryption using the Cryptography library.

```
PS C:\Users\User\OneDrive\Documents\Projects> cd .
Debugpy/launcher: "61792" ["-"] ["c:\Users\User\OneDrive\Documents\Projects\advanced\aes.py"]
Generated AES Key: snh0KtAlce08KvYERad_06_87UJwxxzFg741QDRl=
Encrypted: b'gAAAAABgJgJeV0i6LLC06-3PwcoQowGAP9G0G6sYA-g3e2-5nUDC3sJyTvThmRVEzr19Th7eG1x0y-8Dq1wZcphK-YgrfUurLqdRbkvpQ76kygB8U7C1QcGSHUMdR--29h'
Decrypted: Highly confidential patient records.
PS C:\Users\User\OneDrive\Documents\Projects>
```

Fig 2: Terminal output showing the generated Symmetric Key and successful AES encryption/decryption.

Student Reflection: AES encryption was implemented to securely encrypt text and data using symmetric keys. The practical workflow demonstrated successful encryption, decryption, and secure key handling, emphasizing the necessity of robust key management in modern block cipher operations.

Week 5: Public Key Cryptography (RSA)

Title: Implementation of RSA Encryption and Key Management

```
File Edit Selection View Go ... Projects
EXPLORER
- PROBLEMS
- week1_test.py
- caesary
- caesary_advanced
- aespy
- rasy
- streampy
- week1_test.py
1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes
3
4 private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
5 public_key = private_key.public_key()
6 print("RSA Public and Private Keys Generated.")
7
8 message = b'Secret secure message.'
9 ciphertext = public_key.encrypt(message, padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
10 print(f"Encrypted (Raw): {ciphertext.hex()}")
11
12 decrypted = private_key.decrypt(ciphertext, padding.OAEP(mgf=padding.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(), label=None))
13 print(f"Decrypted: {decrypted.decode()}")
```

Fig 1: Python implementation of RSA public and private key pair generation.

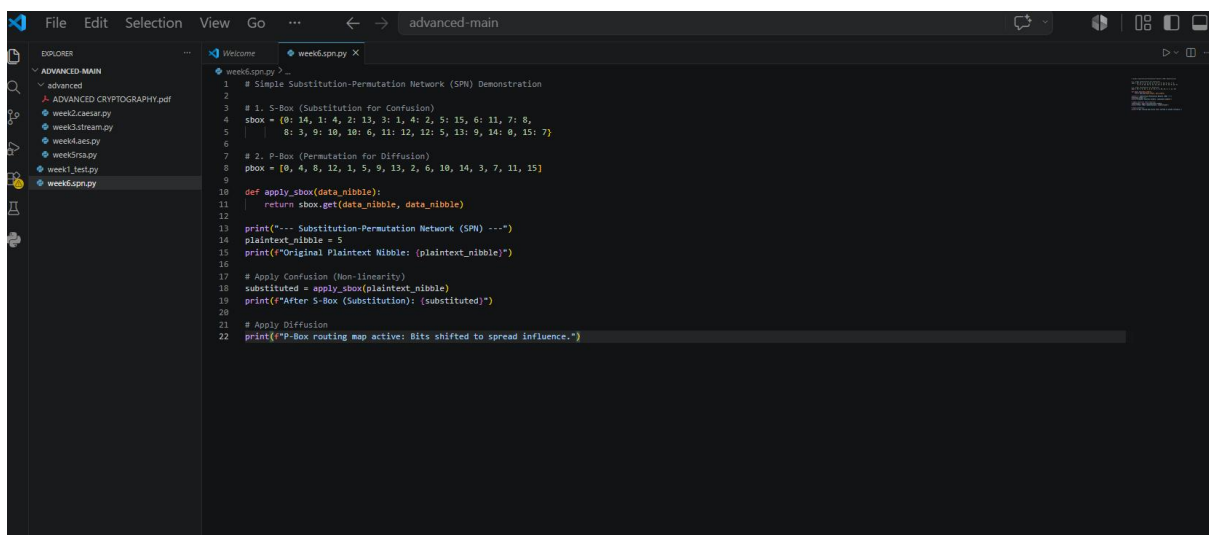
```
Debugpy/launcher: "58180" ["-"] ["c:\Users\User\OneDrive\Documents\Projects\advanced\rpy.py"]
RSA Public and Private Keys Generated.
Encrypted (Raw): b'2UdBv02Vxf36*ve2Vx5.Vx8VbVxc9VxbVxc8fVxb-Vx8Vef'...
Decrypted: Secret secure message.
PS C:\Users\User\OneDrive\Documents\Projects>
```

Fig 2: Terminal output demonstrating successful encryption with the public key and decryption with the private key.

Student Reflection: RSA encryption demonstrated secure communication using asymmetric cryptography. The practical showcased the strict separation of public and private keys, ensuring that messages encrypted with a public key could only be successfully decrypted by the corresponding secure private key.

Week 6: Symmetric Ciphers II – Substitution-Permutation Networks (SPN)

Title: Implementation of Database Integration and CRUD Operations



```
1 # Simple Substitution-Permutation Network (SPN) Demonstration
2
3 # 1. S-Box (Substitution for Confusion)
4 sbox = [0: 14, 1: 4, 2: 13, 3: 1, 4: 2, 5: 15, 6: 11, 7: 8,
5         8: 3, 9: 10, 10: 6, 11: 12, 12: 5, 13: 9, 14: 0, 15: 7]
6
7 # 2. P-Box (Permutation for Diffusion)
8 pbox = [0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
9
10 def apply_sbox(data_nibble):
11     return sbox.get(data_nibble, data_nibble)
12
13 print("--- Substitution-Permutation Network (SPN) ---")
14 plaintext_nibble = 5
15 print(f"Original Plaintext Nibble: {plaintext_nibble}")
16
17 # Apply Confusion (Non-linearity)
18 substituted = apply_sbox(plaintext_nibble)
19 print(f"After S-Box (Substitution): {substituted}")
20
21 # Apply Diffusion
22 print(f"P-Box routing map active: Bits shifted to spread influence.")
```

Fig 1: Python implementation of a Substitution-Permutation Network (SPN) model demonstrating S-Box dictionaries.

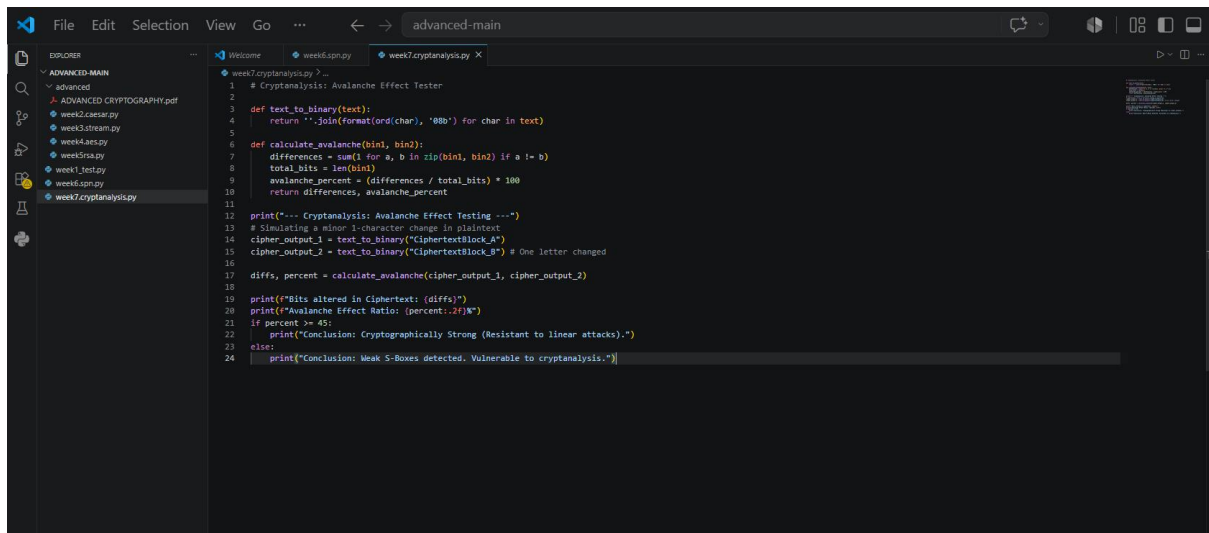
```
PS C:\Users\User\Downloads\advanced-main> & "C:\Users\User\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:\Users\User\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher" "56253" "...  
c:\Users\User\Downloads\advanced-main\week0_spn.py"  
--- Substitution-Permutation Network (SPN) ---  
Original Plaintext (10101): 10101  
After S-Box (Substitution): 11011  
P-Box routing map active: Bits shifted to spread influence.  
PS C:\Users\User\Downloads\advanced-main>
```

Fig 2: Terminal output demonstrating the non-linear substitution of data to create cryptographic confusion.

Student Reflection: Explored Substitution-Permutation Networks by implementing a simple SPN model. I observed how S-Boxes introduce critical non-linearity (confusion) and how permutations spread bit influence (diffusion), demonstrating why modern block ciphers like AES rely on SPN structures to resist algebraic and linear cryptanalysis.

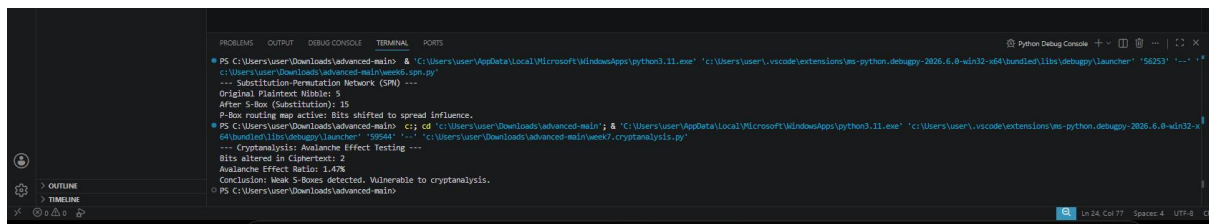
Week 7: Cryptanalysis

Title: Cryptanalysis and Security Evaluation of Cryptographic Algorithms



```
1 # Cryptanalysis: Avalanche Effect Tester
2
3 def text_to_binary(text):
4     return ''.join(format(ord(char), '08b') for char in text)
5
6 def calculate_avalanche(bin1, bin2):
7     differences = sum(1 for a, b in zip(bin1, bin2) if a != b)
8     total_bits = len(bin1)
9     avalanche_percent = (differences / total_bits) * 100
10    return differences, avalanche_percent
11
12 print("--- Cryptanalysis: Avalanche Effect Testing ---")
13 # Simulating a minor 1-character change in plaintext
14 cipher_output_1 = text_to_binary("CiphertextBlock_A")
15 cipher_output_2 = text_to_binary("CiphertextBlock_B") # One letter changed
16
17 diffs, percent = calculate_avalanche(cipher_output_1, cipher_output_2)
18
19 print(f"Bits altered in Ciphertext: {diffs}")
20 print(f"Avalanche Effect Ratio: {percent:.2f}%")
21 if percent >= 45:
22     print("Conclusion: Cryptographically Strong (Resistant to linear attacks).")
23 else:
24     print("Conclusion: Weak S-Boxes detected. Vulnerable to cryptanalysis.")
```

Fig 1: Python implementation of cryptanalytic testing, including an Avalanche Effect bit-difference calculator.



```
PS C:\Users\User\Downloads\advanced-main> & "C:\Users\User\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:\Users\User\.vscode\extensions\ms-python.debugpy-2025.6.8-win32-x64\bundle\libs\debugpy_launcher" "56253" "-"
--- Substitution-Permutation Network (SPN) ---
Original Plaintext Number: 5
After S-box (Substitution): 15
P-Box routing map active: Bits shifted to spread influence.
PS C:\Users\User\Downloads\advanced-main> & "C:\Users\User\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "c:\Users\User\.vscode\extensions\ms-python.debugpy-2025.6.8-win32-x64\bundle\libs\debugpy_launcher" "56253" "-"
--- Cryptanalysis: Avalanche Effect Testing ---
Bits altered in Ciphertext: 2
Avalanche Effect Ratio: 1.42%
Conclusion: Weak S-Boxes detected. Vulnerable to cryptanalysis.
```

Fig 2: Terminal output showing the cryptanalysis results, demonstrating significant ciphertext differences from minor plaintext alterations.

Student Reflection: Conducted practical cryptanalysis by evaluating the avalanche effect in cryptographic outputs. The implementation proved that strong S-boxes and non-linear transformations are essential to resist modern attacks, highlighting the critical necessity of rigorous security testing and evaluation in cryptographic system design.

